Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej
Kierunek: Informatyka i Systemy Inteligentne

System obsługi przesyłek kurierskich i dostaw

Etap 2 (21.X): Architektura systemu, przypadki użycia oraz struktury danych

Przedmiot:	Inżynieria Oprogramowania (Semestr zimowy 2026/2027)
Prowadzący:	dr inż. Marek Zachara
Temat projektu:	Temat nr 3 – Przesyłki kurierskie
Skład zespołu:	Marcel Kosmala, Igor Krzystek, Filip Kontek
Data prezentacji:	21 października 2026 r.

Spis treści

1 Wprowadzenie i główny zakres funkcjonalny **2**
1.1 Cykl życia zamówienia (Automat stanów) . . . 2

2 Architektura systemu **2**
2.1 Opis głównych komponentów i przepływów . . . 3

3 Analiza przypadków użycia (UML Use-Cases) **5**
3.1 Perspektywa Klienta . . . 5
3.2 Perspektywa Dostawcy i Kuriera . . . 6

4 Projekt struktury danych (Baza relacyjna PostgreSQL + PostGIS) **8**
4.1 Kluczowe decyzje projektowe w strukturze bazy danych . . . 8
4.2 Skrypt definicji schematu bazy danych (DDL) . . . 8

5 Optymalizacja lokalizacji kurierów i przydziału (Integracja z Redis) **11**
5.1 Identyfikacja wąskiego gardła architektonicznego . . . 11
5.2 Wprowadzenie struktury in-memory (Redis) . . . 11

1 Wprowadzenie i główny zakres funkcjonalny

Celem projektu jest zaprojektowanie systemu informatycznego wspierającego proces zamawiania oraz dostarczania przesyłek i posiłków. System integruje pracę trzech głównych ról użytkowników, zapewniając dedykowane procesy biznesowe (workflow) oraz kontrolę uprawnień:

- **Klient (`CLIENT`)** – przegląda ofertę dostawców (z możliwością wyszukiwania lokali w pobliżu), kompletuje koszyk, składa i opłaca zamówienia, śledzi status realizacji w czasie rzeczywistym, a po dostarczeniu wystawia ocenę (1–5) wraz z komentarzem dla dostawcy oraz kuriera.
- **Dostawca (`VENDOR`)** – zarządza katalogiem swoich produktów (dodawanie, edycja, ukrywanie pozycji w ofercie), odbiera nowe zamówienia, decyduje o ich akceptacji bądź odrzuceniu oraz aktualizuje stan przygotowania paczki.
- **Kurier (`COURIER`)** – zarządza swoim statusem dostępności, cyklicznie raportuje swoją bieżącą lokalizację GPS, przegląda przydzielone zlecenia, odbiera przygotowane zamówienia z lokalu dostawcy i potwierdza ich dostarczenie do klienta.

1.1 Cykl życia zamówienia (Automat stanów)

Każde zamówienie w systemie jest zarządzane przez deterministyczny automat stanów zapisywany w bazie danych:

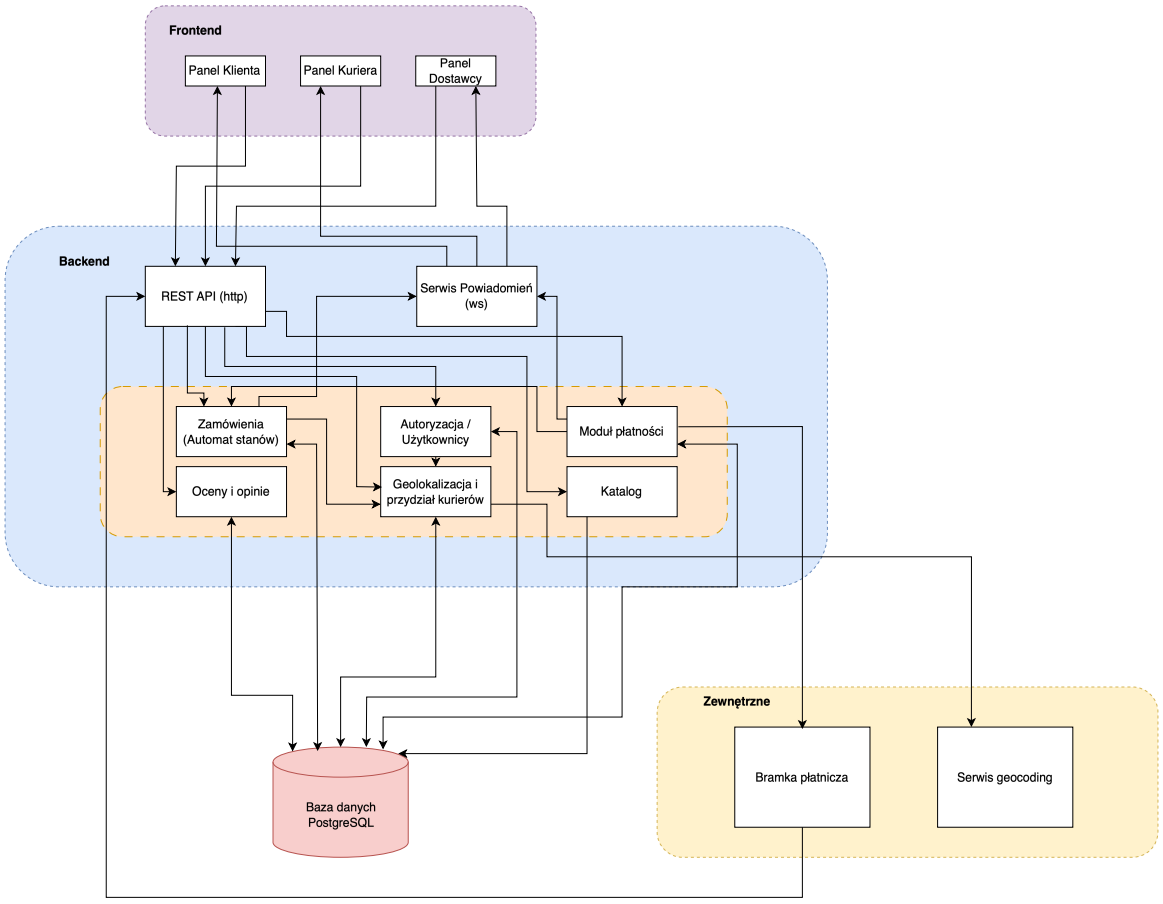
$$\texttt{ORDERED} \rightarrow \texttt{ACCEPTED} \rightarrow \texttt{PREPARATION} \rightarrow \texttt{SHIPPING} \rightarrow \texttt{DELIVERED}$$

Dodatkowo automat obsługuje stany wyjątkowe:

- `CANCELLED` – anulowanie zamówienia przez klienta (możliwe wyłącznie w stanie początkowym `ORDERED`, zanim dostawca rozpocznie realizację),
- `REJECTED` – odrzucenie zamówienia przez dostawcę przed jego przyjęciem do realizacji.

2 Architektura systemu

System został zaprojektowany w oparciu o modularną architekturę trójwarstwową z wyraźnym podziałem na warstwę prezentacji (**Frontend**), warstwę logiki biznesowej (**Backend**), warstwę składowania danych (**PostgreSQL + PostGIS**) oraz integracje zewnętrzne. Ogólny schemat architektury przedstawia Rysunek 1.



Rysunek 1: Logiczna architektura systemu przesyłek kurierskich wraz z przepływem danych między modułami.

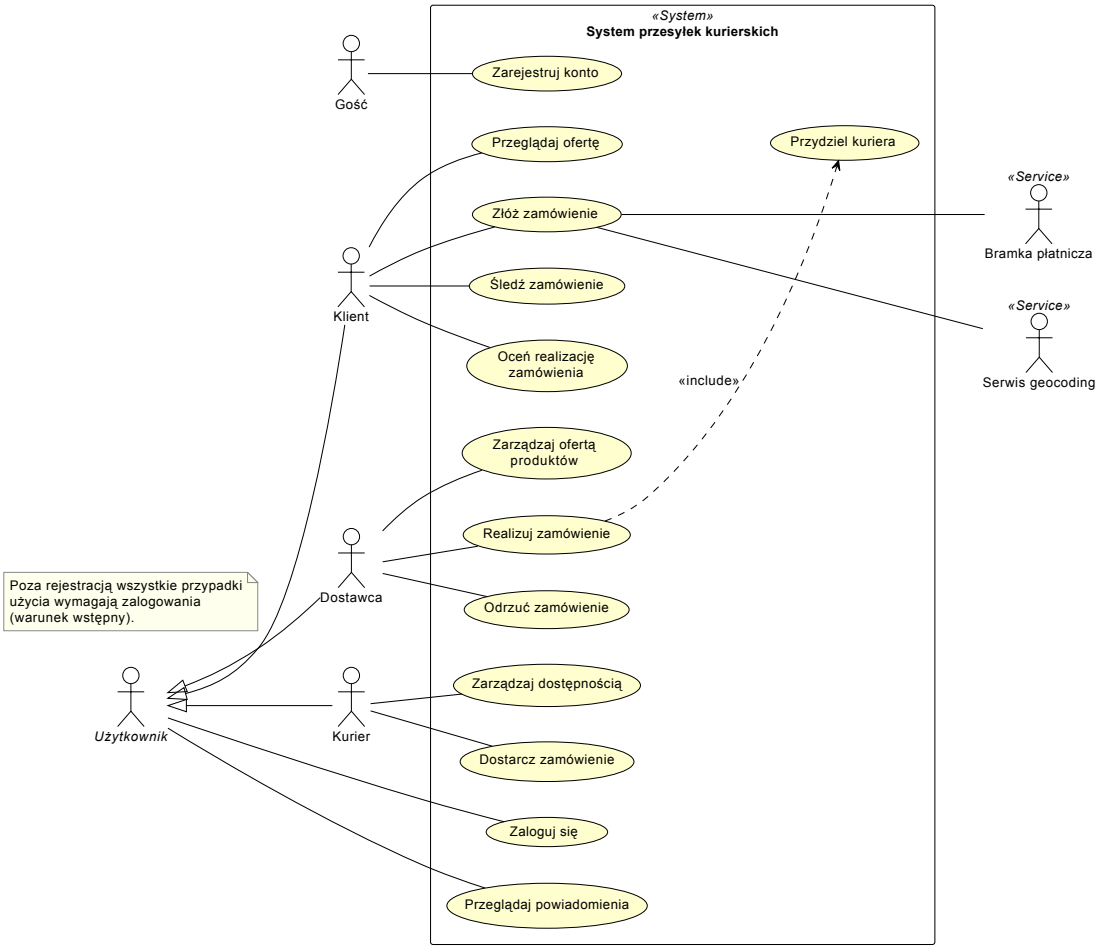
2.1 Opis głównych komponentów i przepływów

1. **Warstwa Frontend:** Podzielona na trzy dedykowane widoki: *Panel Klienta*, *Panel Dostawcy* oraz *Panel Kuriera*. Komunikuje się z serwerem synchronicznie poprzez protokół HTTP (`REST API`) oraz asynchronicznie za pomocą połączenia dwukierunkowego WebSocket (`Serwis Powiadomień (ws)`), co pozwala na natychmiastowe dostarczanie powiadomień o zmianach statusu zamówienia do każdej ze stron.
2. **Moduły domenowe Backendu:**
   - **Zamówienia (Automat stanów)** – centralny moduł koordynujący przejścia między stanami zamówienia, wyzwalający przydział kuriera oraz wysyłkę powiadomień.
   - **Geolokalizacja i przydział kurierów** – odpowiada za zamianę adresów tekstowych na współrzędne geograficzne we współpracy z zewnętrznym *Serwisem geocoding* (np. API Nominatim / OpenStreetMap) oraz za wyszukiwanie najbliższego dostępnego kuriera względem lokalizacji dostawcy za pomocą zapytań przestrzennych PostGIS.
   - **Katalog** – obsługuje operacje CRUD na ofercie produktów poszczególnych dostawców.
   - **Moduł płatności** – integruje system z zewnętrzną *Bramką płatniczą* i odbiera zwrotne potwierdzenia transakcji (webhooki), aktualizując status płatności i powiadamiając o tym odpowiednie strony.
   - **Autoryzacja / Użytkownicy** – zarządza rejestracją, logowaniem, sesjami użytkowników oraz profilami ról.

- **Oceny i opinie** – weryfikuje uprawnienia do wystawienia oceny (tylko dla zamówień ze statusem `DELIVERED`, maksymalnie jedna ocena dla kuriera i jedna dla dostawcy na zamówienie) oraz aktualizuje średnie oceny profilowe.

3 Analiza przypadków użycia (UML Use-Cases)

Model funkcjonalny systemu został podzielony na diagram przeglądowy (Rysunek 2), który definiuje hierarchię aktorów i główne procesy, oraz trzy szczegółowe diagramy prezentujące perspektywę każdej roli.

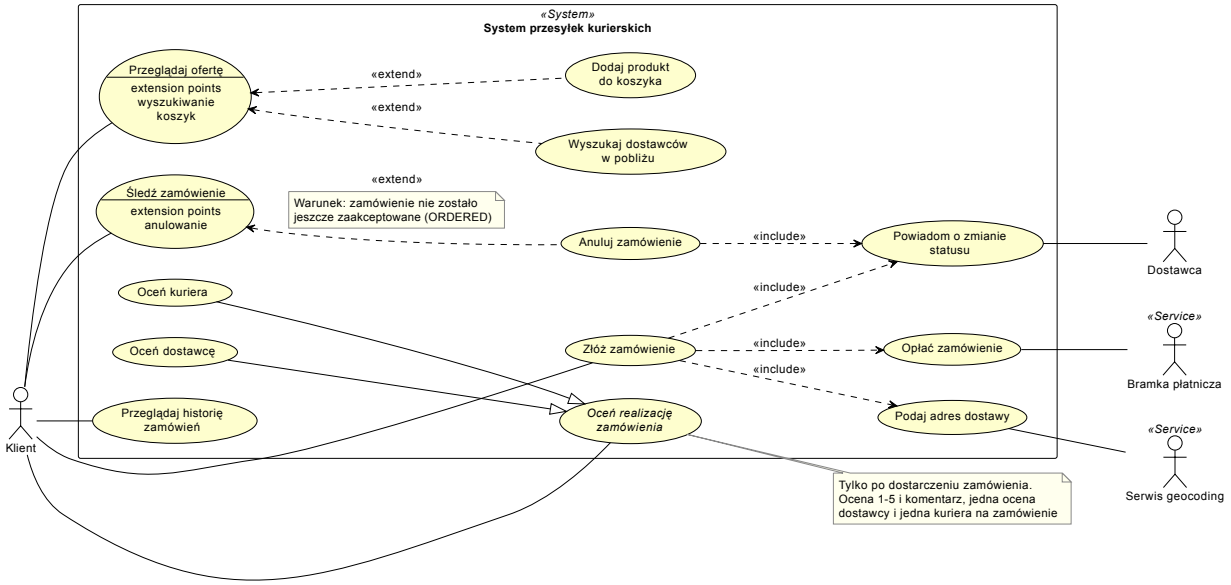


Rysunek 2: Ogólny diagram przypadków użycia systemu przesyłek kurierskich.

Na diagramie ogólnym (Rysunek 2) wyodrębniono niezalogowanego aktora *Gość* (uprawnionego do rejestracji konta) oraz abstrakcyjnego aktora *Użytkownik*, po którym dziedziczą role *Klient*, *Dostawca* i *Kurier*. Wszystkie te role współdzielą przypadki użycia *Zaloguj się* oraz *Przeglądaj powiadomienia*.

3.1 Perspektywa Klienta

Szczegółowe przypadki użycia dla roli Klienta prezentuje Rysunek 3. Podczas przeglądania oferty klient może opcjonalnie (`«extend»`) filtrować dostawców w pobliżu oraz dodawać produkty do koszyka. Złożenie zamówienia bezwarunkowo (`«include»`) wymaga podania adresu dostawy (walidowanego przez zewnętrzny *Serwis geocoding*), opłacenia zamówienia poprzez *Bramkę płatniczą* oraz wysłania powiadomienia do dostawcy.

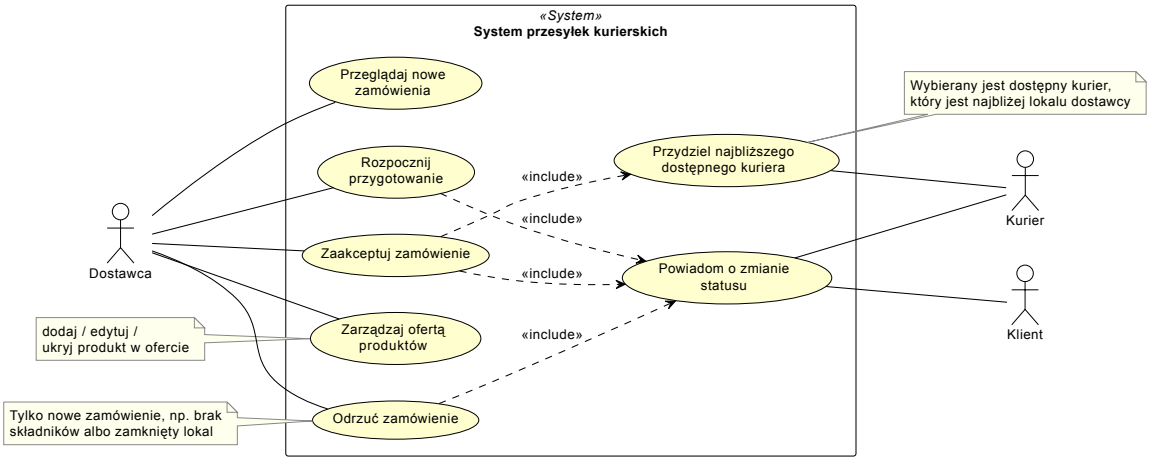


Rysunek 3: Diagram przypadków użycia – perspektywa Klienta.

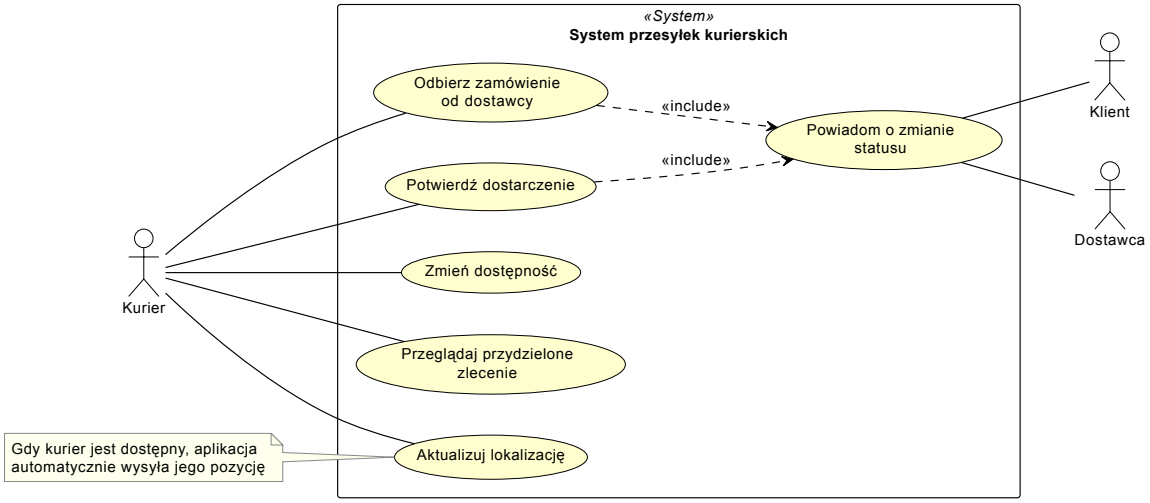
3.2 Perspektywa Dostawcy i Kuriera

Rysunek 4 oraz Rysunek 5 przedstawiają procesy realizowane przez Dostawcę oraz Kuriera:

- **Akceptacja i przydział kuriera:** Zaakceptowanie zamówienia przez dostawcę automatycznie uruchamia przypadek *Przydziel najbliższego dostępnego kuriera* (relacja `«include»`), który wybiera dostępnego kuriera znajdującego się najbliżej lokalu dostawcy i wysyła powiadomienia o zmianie statusu zarówno do Klienta, jak i do Kuriera.
- **Realizacja dostawy i telemetria:** Kurier w trakcie dostępności cyklicznie przesyła swoją pozycję GPS (*Aktualizuj lokalizację*), a przy odbiorze paczki od dostawcy (`SHIPPING`) oraz potwierdzeniu doręczenia (`DELIVERED`) system automatycznie rozsyła powiadomienia do pozostałych uczestników procesu.



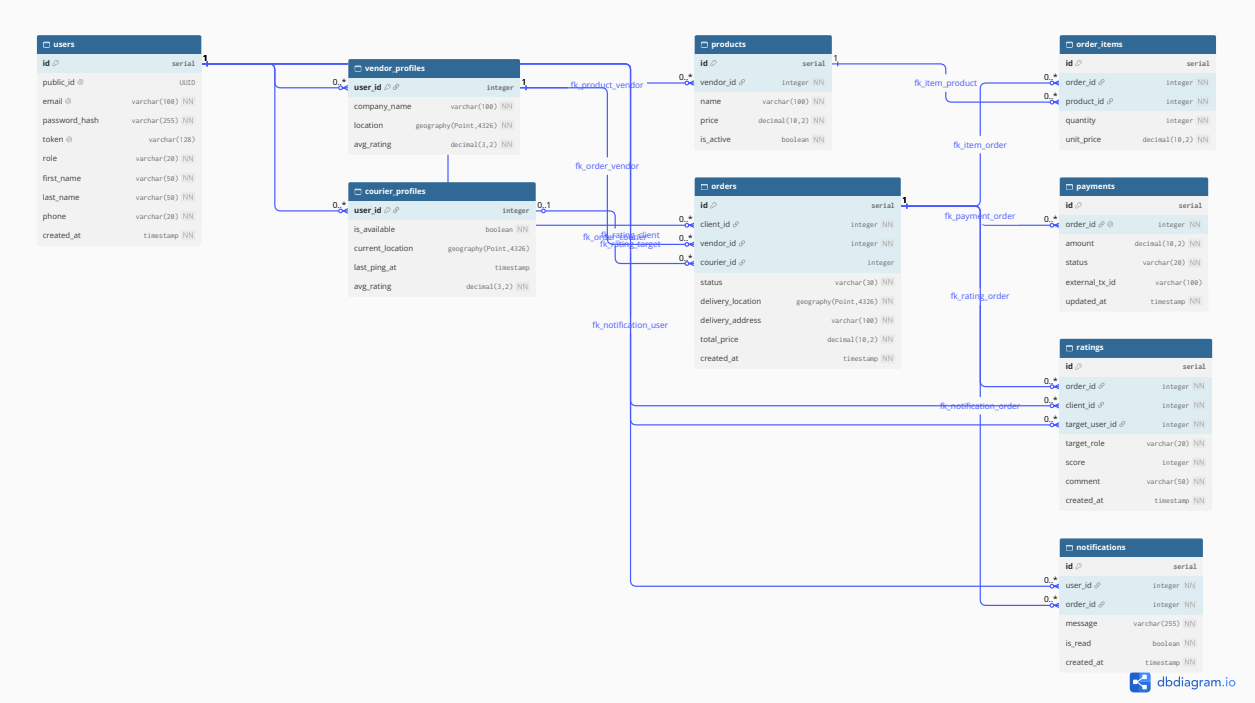
Rysunek 4: Diagram przypadków użycia – perspektywa Dostawcy.



Rysunek 5: Diagram przypadków użycia – perspektywa Kuriera.

4 Projekt struktury danych (Baza relacyjna PostgreSQL + PostGIS)

Dane persystentne składowane są w relacyjnej bazie danych PostgreSQL wzbogaconej o rozszerzenie przestrzenne **PostGIS**. Relacje pomiędzy tabelami oraz krotności powiązań kluczy obcych przedstawia diagram ERD na Rysunku 6.



Rysunek 6: Diagram związków encji (ERD) z krotnościami relacji i kluczami obcymi.

4.1 Kluczowe decyzje projektowe w strukturze bazy danych

- **Obsługa danych geograficznych (`PostGIS`):** Lokalizacja lokalu dostawcy (`vendor_profiles.location`), bieżąca pozycja kuriera (`courier_profiles.current_location`) oraz punkt docelowy zamówienia (`orders.delivery_location`) używają typu `geography(Point, 4326)` (układ współrzędnych WGS 84). Na kolumnie `current_location` założono indeks przestrzenny `GIST` (`idx_courier_loca`) co gwarantuje błyskawiczne wyszukiwanie najbliższego kuriera.
- **Integralność cenowa zamówień:** Tabela `order_items` przechowuje kolumnę `unit_price`, która zamraża cenę produktu w momencie zakupu – późniejsza edycja ceny przez dostawcę w tabeli `products` nie zmienia historycznych kwot zamówień.
- **Więzy spójności ocen (`ratings`):** Zastosowano ograniczenie unikalności `UNIQUE (order_id, target_role)`, dzięki czemu klient w ramach jednego zamówienia może wystawić dokładnie jedną ocenę dostawcy (`VENDOR`) oraz jedną ocenę kurierowi (`COURIER`).
- **Trwałość powiadomień (`notifications`):** Oprócz wysyłki w czasie rzeczywistym przez WebSocket, powiadomienia są zapisywane w bazie wraz z flagą `is_read`, co pozwala użytkownikom na odczytanie historii komunikatów po ponownym zalogowaniu.

4.2 Skrypt definicji schematu bazy danych (DDL)

Poniższy listing prezentuje kompletny kod SQL definiujący tabele, klucze główne i obce, ograniczenia `CHECK` oraz indeksy przestrzenne:

```
1 create extension if not exists postgis;
```

```
2
3  create table users (
4      id serial primary key,
5      public_id UUID default gen_random_uuid() unique,
6      email varchar(100) not null unique,
7      password_hash varchar(255) not null,
8      role varchar(20) not null check (role in ('CLIENT', 'VENDOR', 'COURIER')),
9      first_name varchar(50) not null,
10     last_name varchar(50) not null,
11     phone varchar(20) not null,
12     created_at timestamp not null default current_timestamp
13 );
14
15 create table vendor_profiles (
16     user_id integer primary key references users(id) on delete cascade,
17     company_name varchar(100) not null,
18     location geography(Point, 4326) not null,
19     avg_rating decimal(3,2) not null default 0.00
20 );
21
22 create table courier_profiles (
23     user_id integer primary key references users(id) on delete cascade,
24     is_available boolean not null default false,
25     current_location geography(Point, 4326),
26     last_ping_at timestamp,
27     avg_rating decimal(3,2) not null default 0.00
28 );
29
30 create table products (
31     id serial primary key,
32     vendor_id integer not null,
33     name varchar(100) not null,
34     price decimal(10,2) not null check (price >= 0),
35     is_active boolean not null default true,
36     constraint fk_product_vendor
37         foreign key (vendor_id) references vendor_profiles(user_id) on delete
           cascade
38 );
39
40 create table orders (
41     id serial primary key,
42     client_id integer not null,
43     vendor_id integer not null,
44     courier_id integer,
45     status varchar(30) not null default 'ORDERED' CHECK (
46         status in ('ORDERED', 'ACCEPTED', 'PREPARATION', 'SHIPPING', 'DELIVERED', '
           REJECTED', 'CANCELLED')
47     ),
48     delivery_location geography(Point, 4326) not null,
49     delivery_address varchar(100) not null,
50     total_price decimal(10, 2) not null check (total_price >= 0),
51     created_at timestamp not null default current_timestamp,
52     constraint fk_order_client
53         foreign key (client_id) references users(id) on delete restrict,
54     constraint fk_order_vendor
55         foreign key (vendor_id) references vendor_profiles(user_id) on delete
           restrict,
56     constraint fk_order_courier
57         foreign key (courier_id) references courier_profiles(user_id) on delete set
            null
58 );
59
60 create table order_items (
61     id serial primary key,
62     order_id integer not null,
```

```
    product_id integer not null,
    quantity integer not null check (quantity > 0),
    unit_price decimal(10,2) not null check (unit_price >= 0),
    constraint fk_item_order
        foreign key (order_id) references orders(id) on delete cascade,
    constraint fk_item_product
        foreign key (product_id) references products(id) on delete restrict
);

create table payments (
    id serial primary key,
    order_id integer not null unique,
    amount decimal(10,2) not null check (amount >= 0),
    status varchar(20) not null default 'PENDING' check (
        status in ('PENDING', 'COMPLETED', 'FAILED')
    ),
    external_tx_id varchar(100),
    updated_at timestamp not null default current_timestamp,
    constraint fk_payment_order
        foreign key (order_id) references orders(id) on delete cascade
);

create table ratings (
    id serial primary key,
    order_id integer not null,
    client_id integer not null,
    target_user_id integer not null,
    target_role varchar(20) not null check (target_role in ('VENDOR', 'COURIER')),
    score integer not null check (score between 1 and 5),
    comment varchar(50) not null,
    created_at timestamp not null default current_timestamp,
    constraint fk_rating_order
        foreign key (order_id) references orders(id) on delete cascade,
    constraint fk_rating_client
        foreign key (client_id) references users(id) on delete cascade,
    constraint fk_rating_target
        foreign key (target_user_id) references users(id) on delete cascade,
    constraint uq_order_target_rating unique (order_id, target_role)
);

create table notifications (
    id serial primary key,
    user_id integer not null,
    order_id integer not null,
    message varchar(255) not null,
    is_read boolean not null default false,
    created_at timestamp not null default current_timestamp,
    constraint fk_notification_user
        foreign key (user_id) references users(id) on delete cascade,
    constraint fk_notification_order
        foreign key (order_id) references orders(id) on delete cascade
);

create index idx_courier_location on courier_profiles using gist (current_location)
    ;
```

5 Optymalizacja lokalizacji kurierów i przydziału (Integracja z Redis)

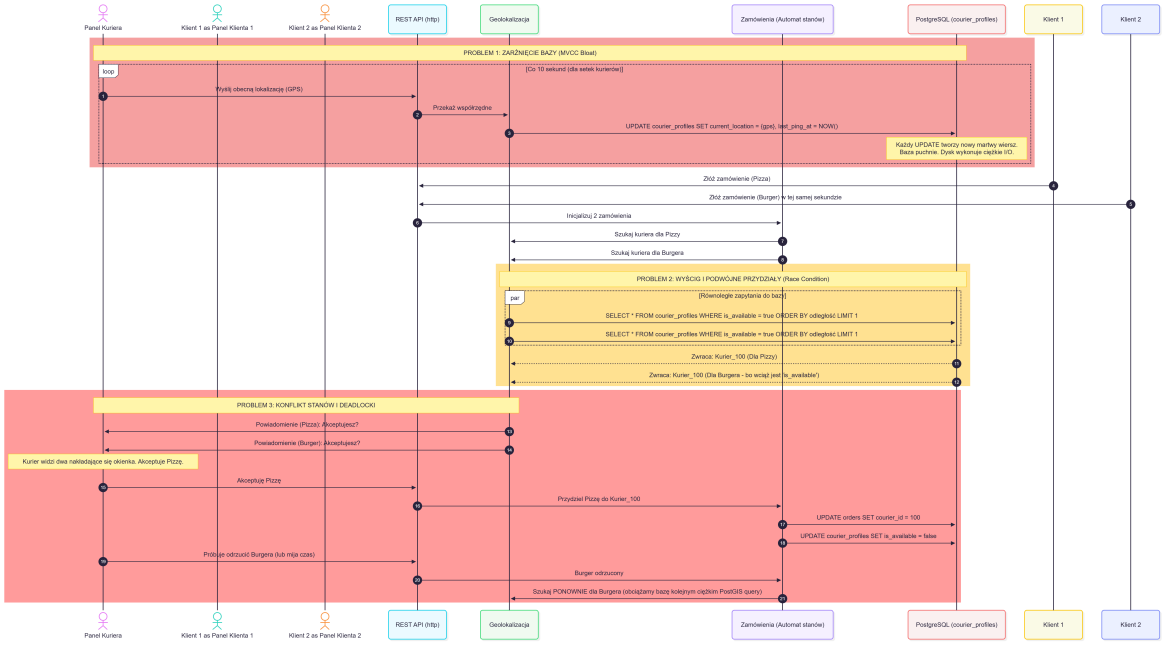
W trakcie prac projektowych nad modułem **Geolokalizacji i przydziału kurierów** zdiagnozowano potencjalne wąskie gardła związane z wydajnością, gdy system zostanie uruchomiony dla dużej floty kurierów w obrębie całego miasta. Problem dotyczy częstotliwości aktualizacji lokalizacji na żywo.

5.1 Identyfikacja wąskiego gardła architektonicznego

Pierwotne założenie, odwzorowane w tabeli `courier_profiles`, zakładało, że aplikacja mobilna kuriera co 10 sekund wysyła do serwera swoją aktualną pozycję (GPS), co skutkowało operacją aktualizacji w głównej bazie relacyjnej: `UPDATE courier_profiles SET current_location = <gps>, last_ping_at = NOW()`.

Takie rozwiązanie w środowisku produkcyjnym niesie za sobą poważne konsekwencje (Rysunek 7):

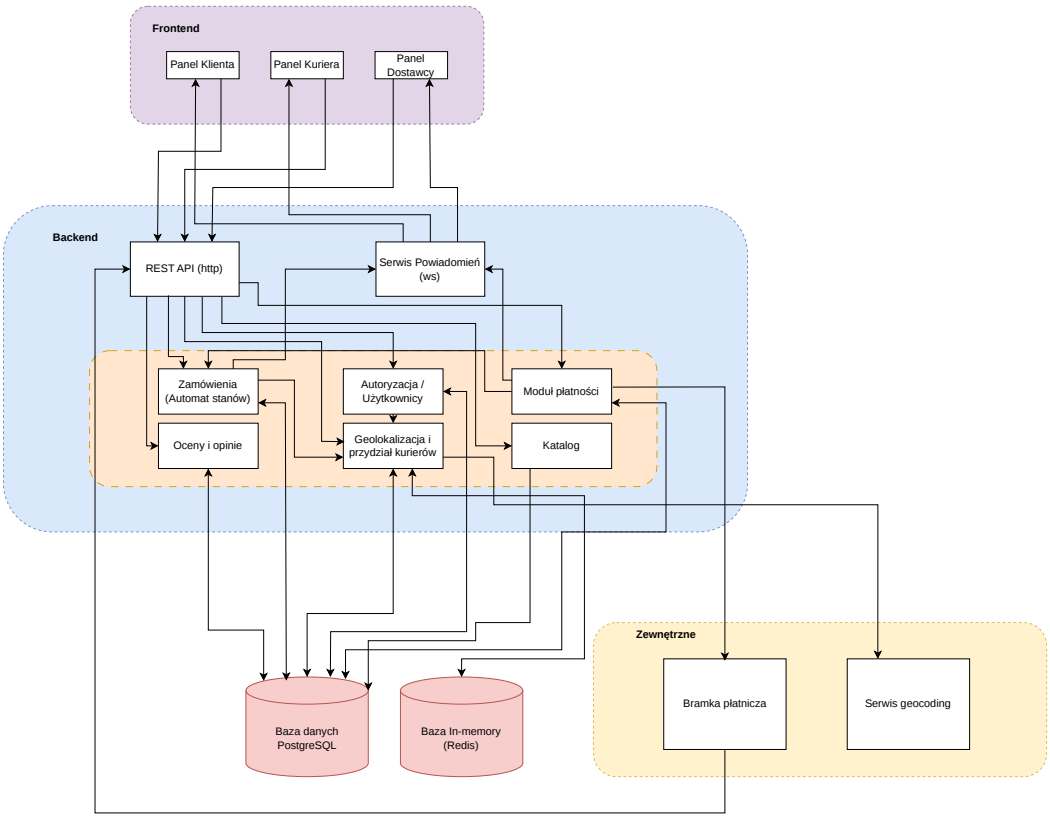
- **Obciążenie wejścia/wyjścia (I/O) i pęcznienie bazy (Table Bloat):** Silnik PostgreSQL wykorzystuje mechanizm MVCC (Multi-Version Concurrency Control). Każda operacja `UPDATE` nie nadpisuje starych danych bezpośrednio na dysku, lecz oznacza stary wiersz jako martwy i tworzy nowy. Dla 200 kurierów wysyłających lokalizację co 10 sekund, generujemy 1200 zapytań `UPDATE` na minutę do jednej tabeli. Spowalnia to zapytania analityczne i inne procesy logiki biznesowej.
- **Konflikty dostępu (Race Conditions):** Jeśli w tym samym momencie do systemu spłyną dwa zamówienia dla lokali w bliskiej odległości, dwa współbieżne zapytania `SELECT` wybiorą tego samego kuriera (ponieważ posiada on wciąż status `is_available = true`).



Rysunek 7: Diagram sekwencyjny uwydatniający problemy z wydajnością i współbieżnością w naiwnym podejściu do przydziału kuriera (tylko PostgreSQL).

5.2 Wprowadzenie struktury in-memory (Redis)

W celu odciążenia głównej bazy danych zaimplementowano hybrydowe rozwiązanie oparte na in-memory datastore – **Redis**. Wykorzystano w nim specjalistyczne moduły do operacji przestrzennych. Zaktualizowaną architekturę systemu przedstawia Rysunek 8.

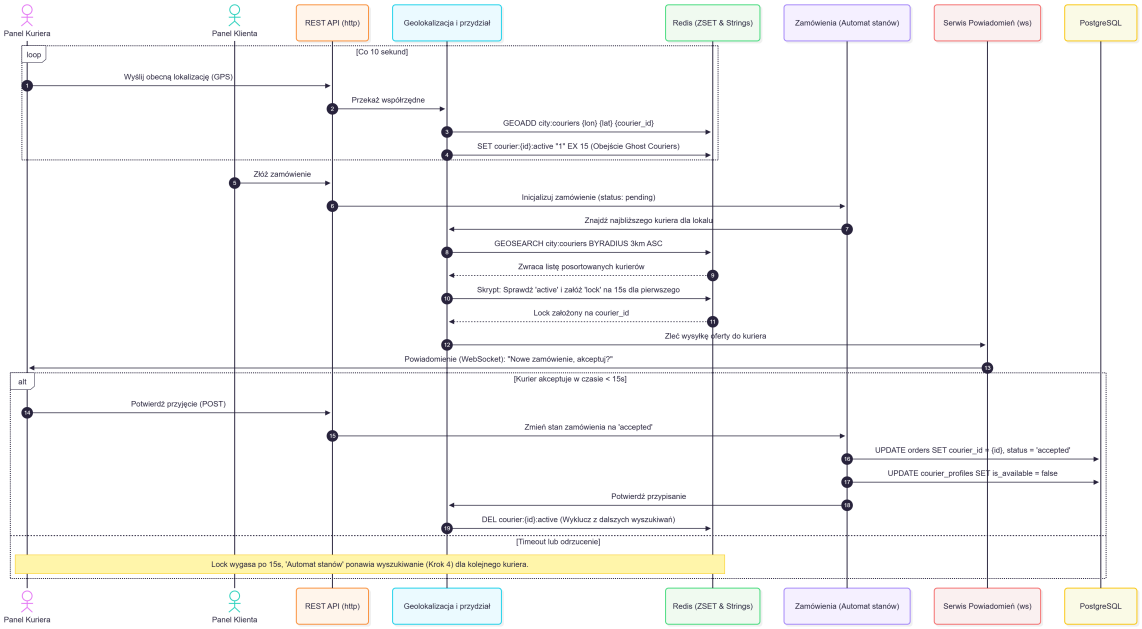


Rysunek 8: Zaktualizowana logiczna architektura systemu uwzględniająca bazę danych Redis dla operacji na żywo.

Działanie nowego, zoptymalizowanego mechanizmu:

1. **Rejestrowanie pozycji (Geohash):** Pingi geolokalizacyjne od kurierów nie uderzają już bezpośrednio w PostgreSQL. Są one asynchronicznie zapisywane w Redisie za pomocą komendy `GEOADD`, która w locie przelicza koordynaty trygonometryczne na algorytm Geohash i zachowuje je w zbiorach posortowanych w pamięci RAM.
2. **Ghost Couriers i Time-to-Live (TTL):** Aby uniknąć przydzielania zamówień kurierom, którzy zgubili zasięg, dodano równoległy zapis unikalnego klucza (np. `SET courier:100:active "1"EX 15`) z ustawionym 15-sekundowym czasem wygaśnięcia.
3. **Soft Locking (Blokady współbieżne):** Podczas przydzielania kuriera za pomocą `GEOSEARCH`, system wykorzystuje krótki skrypt (np. w języku Lua), który atomowo sprawdza dostępność kuriera i tymczasowo go blokuje (soft lock). Gwarantuje to uniknięcie zjawiska podwójnego przydziału.
4. **Ostateczne utrwalenie stanu:** Dopiero gdy kurier zaakceptuje zamówienie za pomocą aplikacji, system informuje o tym Automat Stanów, dokonując *twardego* zapisu do relacyjnej bazy PostgreSQL (aktualizacja kolumny `courier_id` w zamówieniu).

Zoptymalizowany przepływ danych, wolny od rywalizacji o zasoby bazodanowe i wąskich gardeł transakcyjnych, ukazuje Rysunek 9.



Rysunek 9: Zoptymalizowany przepływ przydziału kuriera przy wykorzystaniu zapytań geoprzestrzennych oraz soft-locków w bazie Redis.